BEDROCK LANGUAGE & COMPILER PROJECT

[LOCAL SCIENCE FAIR NAME] · [YEAR]

BedRock: A Hand-Built Compiled Language for Bare-Metal Microcontroller Programming

Karim Mohamed Hanafy El Shamy

[Your School Name]
Category: Systems Software

Abstract

BedRock is a from-scratch compiled programming language and compiler, written in Rust, with native code-generation backends for RISC-V, ARM, MIPS, and x86-64. This project uses BedRock to implement a *direct boot* scheme on the ESP32-C3 microcontroller that bypasses the vendor bootloader chain entirely, giving a BedRock-compiled binary full control of the chip from power-on. The poster documents the compiler's architecture, the reverse-engineered boot-image format direct boot requires, and a demonstration firmware stack — including a Morse-code input decoder and a gesture-locked bootloader gate — built and validated on real hardware with no underlying operating system.

Background

Most embedded development relies on a vendor SDK (such as Espressif's ESP-IDF) or a hosted language runtime that hides the boot process, memory layout, and machine code a program compiles down to. Understanding how a microcontroller actually starts executing code — before any SDK is involved — requires owning the whole stack: the language, the compiler that lowers it to machine code, and the exact bytes that get written to flash.

BedRock began as a general-purpose compiled language with its own compiler, `brc3`. This project extends it to real embedded hardware, treating the microcontroller as a target with no operating system and no safety net.

Research Question & Engineering Goal

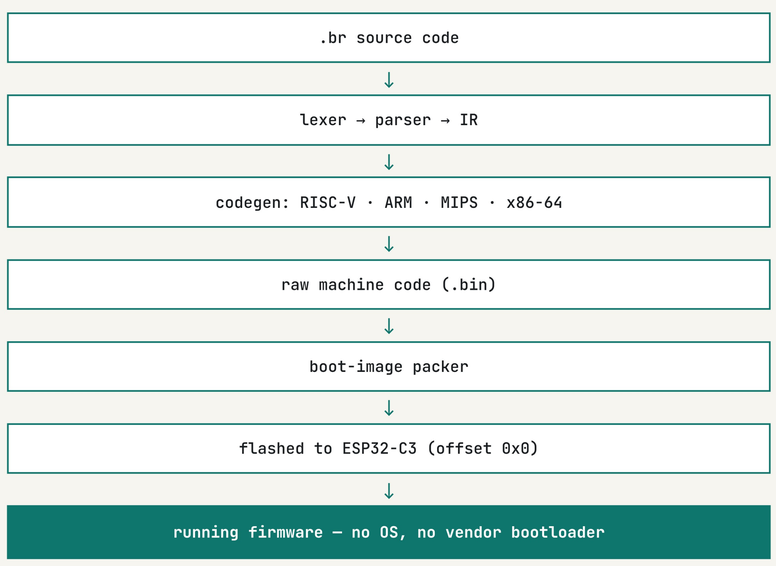
Research question: Can a from-scratch, independently written compiled language generate working bare-metal firmware for a real microcontroller — including full control of the boot process — without relying on a vendor SDK or operating system?

- → Build a compiler with native machine-code backends for real hardware targets.
- → Design a boot-image format and flashing pipeline that bypasses the vendor bootloader.
- → Demonstrate working firmware exercising memory, GPIO, and serial I/O — written entirely in BedRock.

Methodology

1. Design the language and compiler — lexer, parser, and intermediate representation (IR) — in Rust.
2. Build a RISC-V code-generation backend targeting the ESP32-C3.
3. Reverse-engineer the ESP32-C3 ROM's boot-image requirements from the `esptool` source.
4. Write a packing tool that wraps raw compiler output in a flashable boot image.
5. Flash to real hardware and iterate, reading serial output and crash register dumps to localize failures.
6. Build demonstration applications entirely in BedRock to validate the pipeline end-to-end.

System & Boot Pipeline



Results

7,772	4	4
lines of Rust in the compiler	native code-gen targets	demo firmware programs on real hardware

- → Diagnosed and fixed real boot failures — a missing base load address, and missing stack/data addresses that defaulted into read-only flash-mapped memory.
- → Morse-code decoder: reads button press/hold timing, decodes letters, and streams them over USB serial — running stably on hardware.
- → Watchdog-disable library, reusable via BedRock's `include` system, used across every demo program.
- → Gesture-locked bootloader gate: a 3-tap-then-hold sequence must be entered correctly before the board unlocks — the first building block of a planned multi-app firmware.

Discussion

Bare-metal boot failures are hard to localize because very different root causes — an invalid boot-image header, a missing load address, a corrupted stack — all surface identically as a reset loop. Reading raw RISC-V exception register dumps made it possible to tell "the chip never started my code" apart from "my code started, then crashed," which in turn pointed to the real fix each time. The work also showed a practical trade-off: hand-rolled binary formats are flexible but error-prone, while reusing vendor tooling for correctness-critical steps (like the boot-image checksum) is more reliable.

Conclusion & Future Work

BedRock successfully compiles original source programs that run directly on ESP32-C3 hardware, with no operating system and no vendor bootloader involved — demonstrating that a single developer can build a working compiler-to-silicon pipeline end-to-end.

Future work is the **BR3 Micro-Kernel**: a unified firmware hosting several BedRock applications side by side, with per-application memory isolation using RISC-V Physical Memory Protection, the gesture-locked gate already built here as its entry point, and a BedRock-written PC installer that uploads program source — not precompiled binaries — to be interpreted on-device.

Acknowledgments

Thank you to [MENTOR / TEACHER NAME] for guidance throughout this project, and to the [LOCAL SCIENCE FAIR NAME] organizers and judges for their time. BedRock's language, compiler, and firmware were designed and built independently.

References

Espressif Systems, *ESP32-C3 Technical Reference Manual*. · Espressif Systems, *esptool* source and documentation. · *The RISC-V Instruction Set Manual*, RISC-V International.

[QR CODE]

`bedrock.abrdns.com`
`project docs & source`